



MI3

**i** This presentation has live captioning

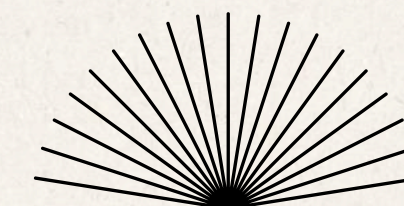
# **CRYPTOGRAPHY APPLIED TO ELECTRONIC VOTING**

**RSA · Blind Signatures · Hash Functions · Digital Signatures**

**PRESENTED BY:**

**DAHMOUN Mouaine Aymen  
BENCHENINA Shouhaib Meheni  
BENZINA Mohamed  
MEZAACHE Abdessamie  
FANTAZI Sami Ilyes**

**PRESENTED TO:  
Ms. KHERROUBI**





# Agenda

|    |                                              |
|----|----------------------------------------------|
| 01 | Introduction to Electronic Voting            |
| 02 | Core Security Requirements                   |
| 03 | Theoretical Foundation: The RSA Cryptosystem |
| 04 | Digital Signatures and Hash Functions        |
| 05 | The Blind Signature Scheme                   |
| 06 | Distributed Trust Architecture               |
| 07 | Dual-Code Voter Scheme (N1 and N2)           |
| 08 | Phase 1: Election Preparation                |
| 09 | Phase 2: The Voting Process                  |
| 10 | Phase 3: Counting and Verification           |
| 11 | Comprehensive Security Analysis              |
| 12 | Python Implementation and Deployment         |
| 13 | Project Conclusion and Insights              |



# 1. Introduction to Electronic Voting

03/15

**01** Electronic voting represents a significant evolution in democratic participation, offering benefits like increased convenience, accessibility, and speed compared to traditional paper-based methods

**02** this digitalization introduces severe security concerns, such as the potential for large-scale duplication, alteration, or fabrication of ballots.

**03** This project develops a cryptographic solution using blind signatures to address these challenges while ensuring a secure and transparent election process.



## 2. Core Security Requirements

04/15

A robust e-voting system must satisfy several critical and often conflicting requirements:

- **core components**

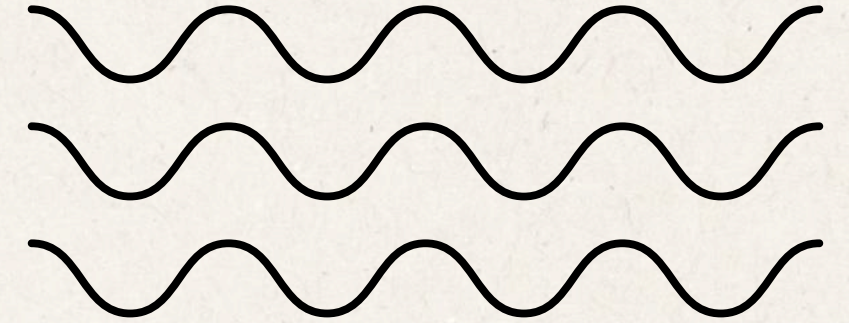
**authenticity** (only eligible citizens vote), **uniqueness** (each citizen votes once), and **anonymity** (votes cannot be linked to voters)

- **core components**

it must guarantee **integrity** so ballots cannot be altered, **verifiability** so voters can confirm their counts, and overall **security** against both internal and external fraud.



### 3. Theoretical Foundation: The RSA Cryptosystem



The protocol is built on **RSA**, a system that uses math to keep information safe. It works by using a public key for everyone to see and a private key that stays secret.

For this project, instead of using huge, complex numbers, we use small numbers to make it easier to learn. These are set in **config.py**:

- **Administrator key:**  $N = 55$ ,  $e = 27$ ,  $d = 3$
- **Counter key:**  $N = 583$ ,  $e = 3$ ,  $d = 347$

These values allow the protocol to show exactly how a voter hides their choice (blinding), how the admin signs it, and how the vote is finally encrypted for the counter to read.



# 4. Digital Signatures and Hash Functions

The helper module **app/crypto/rsa\_utils.py** handles the basic RSA operations like signing and verification, which allow the administrator to certify a ballot without seeing its content. To keep voter codes secure, the project uses a Toy **Tetragraph Hash (TTH)** to create unique fingerprints.

This custom hash works by converting alphanumeric input into a 4-character uppercase code. Because TTH is a one-way function, the commissioner only needs to store the fingerprint in the registry. This ensures that the original codes stay hidden and cannot be reversed, preventing anyone from creating fake ballots



## 5. The Blind Signature Scheme

**The Concept:** To ensure anonymity, we use a scheme where the Administrator authenticates a message without knowing its content. It is analogous to signing a document through an envelope: the signature is valid on the paper inside, but the signer never saw the text.

**The Mathematical Workflow:** The logic, implemented in `app/crypto/blind_signature.py`, follows these steps using the Administrator's RSA public key  $(e, N)$ :

**Blinding:**  $(m, k, e, N) \rightarrow m' = m \cdot k^e \pmod{N}$   
[Voter hides ballot]

**Signing:**  $(m', d, N) \rightarrow m'' = (m')^d \pmod{N}$   
[Admin signs blindly]

**Unblinding:**  $(m'', k, N) \rightarrow s = m'' \cdot k^{-1} \pmod{N}$   
[Voter removes mask]



# 6. Distributed Trust Architecture

RATHER THAN A CENTRAL AUTHORITY, TRUST IS DISTRIBUTED ACROSS FOUR AUTONOMOUS SERVERS. THIS DESIGN ENSURES THAT NO SINGLE ENTITY HAS ENOUGH INFORMATION TO COMMIT UNDETECTED FRAUD OR COMPROMISE VOTER PRIVACY.

## The Four Pillars of the Protocol:

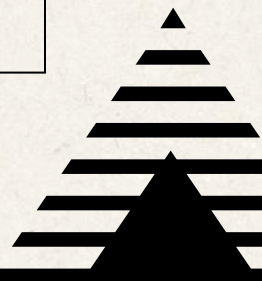
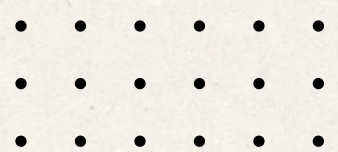
**Commissioner (/commissioner.py):** The Gatekeeper. Manages the list of valid identity codes **N\_1** and the TTH hashes (fingerprints) of **N\_2**

**Administrator (/administrator.py):** The Notary. Validates the voter's right to participate and provides the Blind RSA Signature to authenticate the ballot without seeing the choice.

**Anonymizer (/anonymizer.py):** The Digital Urn. Receives encrypted ballots, verifies **N\_1** validity with the Commissioner, and stores them without any link to the voter's identity.

**Counter (/counter.py):** The Auditor. Decrypts the ballots using its private key, verifies the Administrator's signature, and checks the **N\_2** code before publishing the final results.

08/15





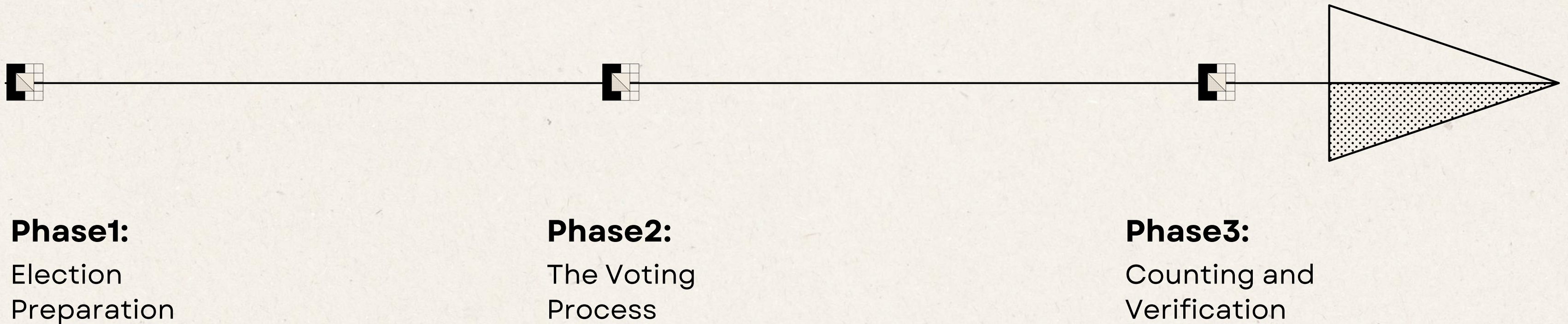
## 7. Dual-Code Voter Scheme (N1 and N2)

Each voter is issued a card with two distinct 12-character alphanumeric codes: N1 and N2. The N1 code proves eligibility and is permanently struck from the list upon use to prevent double-voting. The N2 code is embedded in the ballot for anonymity; the commissioner only stores its TTH hash, which prevents the fabrication of valid ballots while allowing voters to verify their vote post-election.

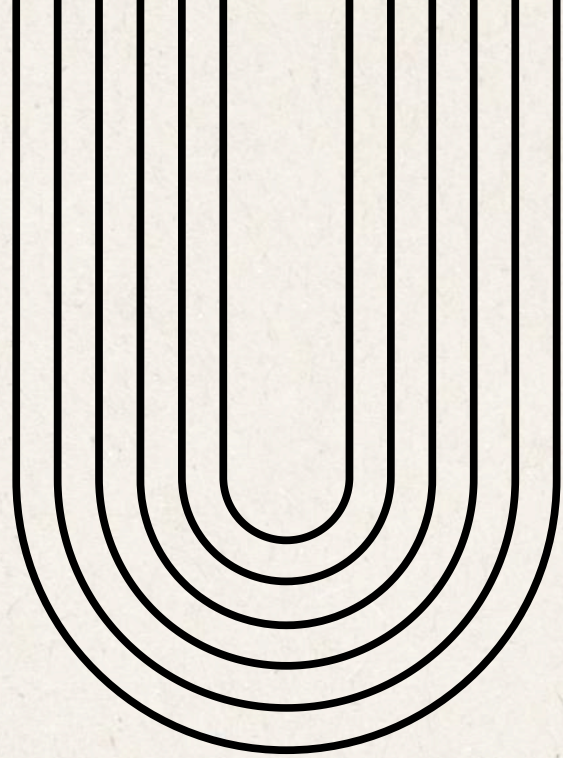


# Complete Voting Protocol

It goes through 3 phases:







## 8. Phase1: Election Preparation

The election setup starts by creating the election state and generating the necessary RSA key pairs for the administrator and the counter. During this phase, the system generates a set number of voter cards containing **N1** and **N2** codes.

To keep the process secure, the commissioner only receives the **N1** codes and the hashed fingerprints of the **N2** codes; the original list of **N2** codes is then destroyed so they cannot be reversed. Finally, the system sets the election status to "**open**" and records the start time. The setup screen displays the election name, the code table, and the generated voter list.



# 9. Phase 2: The Voting Process

To start the voting phase, a voter enters their **N1** code and a grade from 1 to 10 on the voter page. The server first confirms the election is open and that the N1 code is valid and hasn't been used yet.

Once verified, the voter creates a ballot with their choice, their **N2** code, and a random number. This package is "blinded" with a factor **k** and sent to the administrator to be signed. After the administrator signs it, the voter "unblinds" the signature and checks it for accuracy.

The system then hashes the **N2** code using TTH to match it against the commissioner's list. Finally, the grade is encrypted with the counter's public key and sent anonymously to the anonymizer box. This process ensures the administrator certifies the ballot without ever seeing the voter's actual choice.



## 10. Phase 3: Counting and Verification

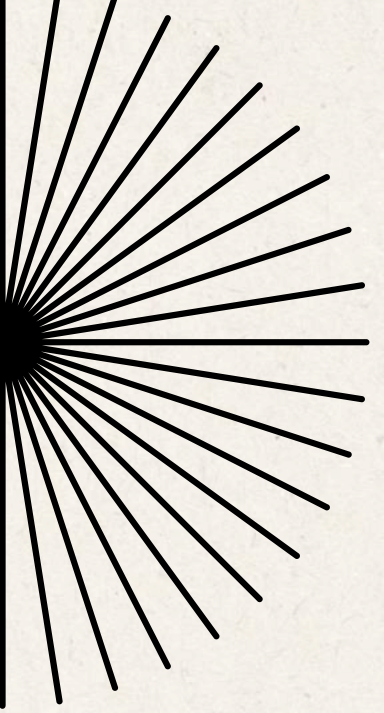
Once the election closes, the counting process begins. The Counter uses its private key to decrypt every ballot and verify the administrator's digital signature. To ensure the vote is legitimate, the system hashes the **N2** code and checks it against the commissioner's list of valid fingerprints.

A ballot is only counted as valid if it passes all these security checks. After the tally is finished, the results are published on a results page that shows:

- The total number of counted and valid ballots.
- The distribution of grades **(1 to 10)** and the average score.
- A list of **(N2, vote)** pairs.

By publishing this list, every voter can use their private N2 code to look up their entry and confirm their vote was recorded correctly without revealing their identity to others.

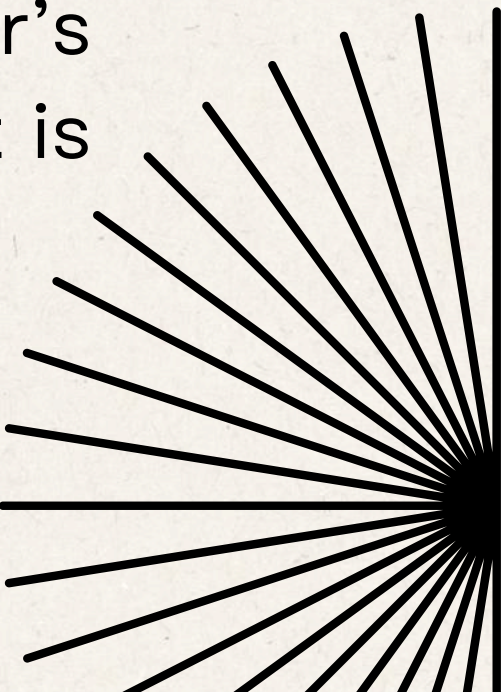




# 11. Comprehensive Security Analysis

The protocol is highly secure because guessing **N1** or **N2** codes is nearly impossible. It prevents fraud by splitting roles: the administrator never sees the vote, and the "**anonymizer**" ensures choices can't be linked to voters. Double voting is also blocked by marking **N1** codes as "**used.**"

As a teaching project, the math uses small numbers so students can follow the calculations by hand. For demonstration, the database stores raw **N2** codes, and the system catches invalid ballots by verifying the administrator's signature and the **N2** fingerprint. This maintains a clear "chain of trust" that is easy to study.





# 12. Python Implementation and Deployment

The project is a Flask-based web application organized into modular sections: **app/crypto** handles the math, and **app/routes** manages the voting steps. It uses **SQLAlchemy** to store data and automatically sets up database tables when it starts.

The application is built to be a clear cryptography demo, with separate files for **RSA**, blind signatures, and the **TTH hash**. A live version lets users experience the full process—from hiding their vote to seeing the final audit list. This clean structure makes it easy to present and follow how each part of the protocol works.



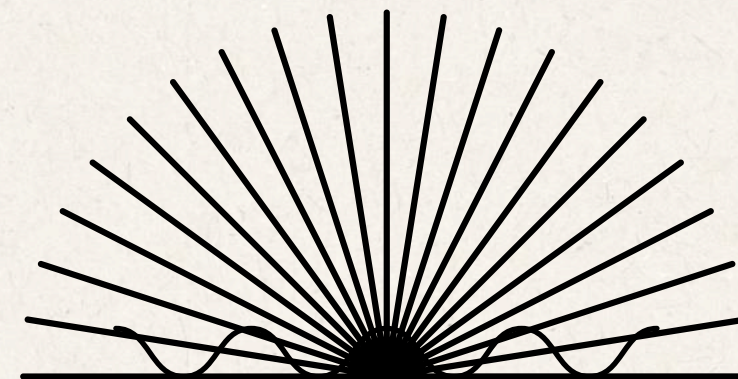
# 13. Project Conclusion and Insights

THIS PROJECT DEMONSTRATES THAT COMBINING RSA ENCRYPTION, BLIND SIGNATURES, AND TTH HASHING CREATES A VOTING SYSTEM THAT IS SIMULTANEOUSLY SECURE, ANONYMOUS, AND VERIFIABLE. BY DISTRIBUTING TRUST ACROSS FOUR INDEPENDENT ENTITIES, THE PROTOCOL ENSURES THAT ELECTION FRAUD WOULD REQUIRE IMPOSSIBLE MULTI-PARTY COLLUSION.

## Lessons Learned:

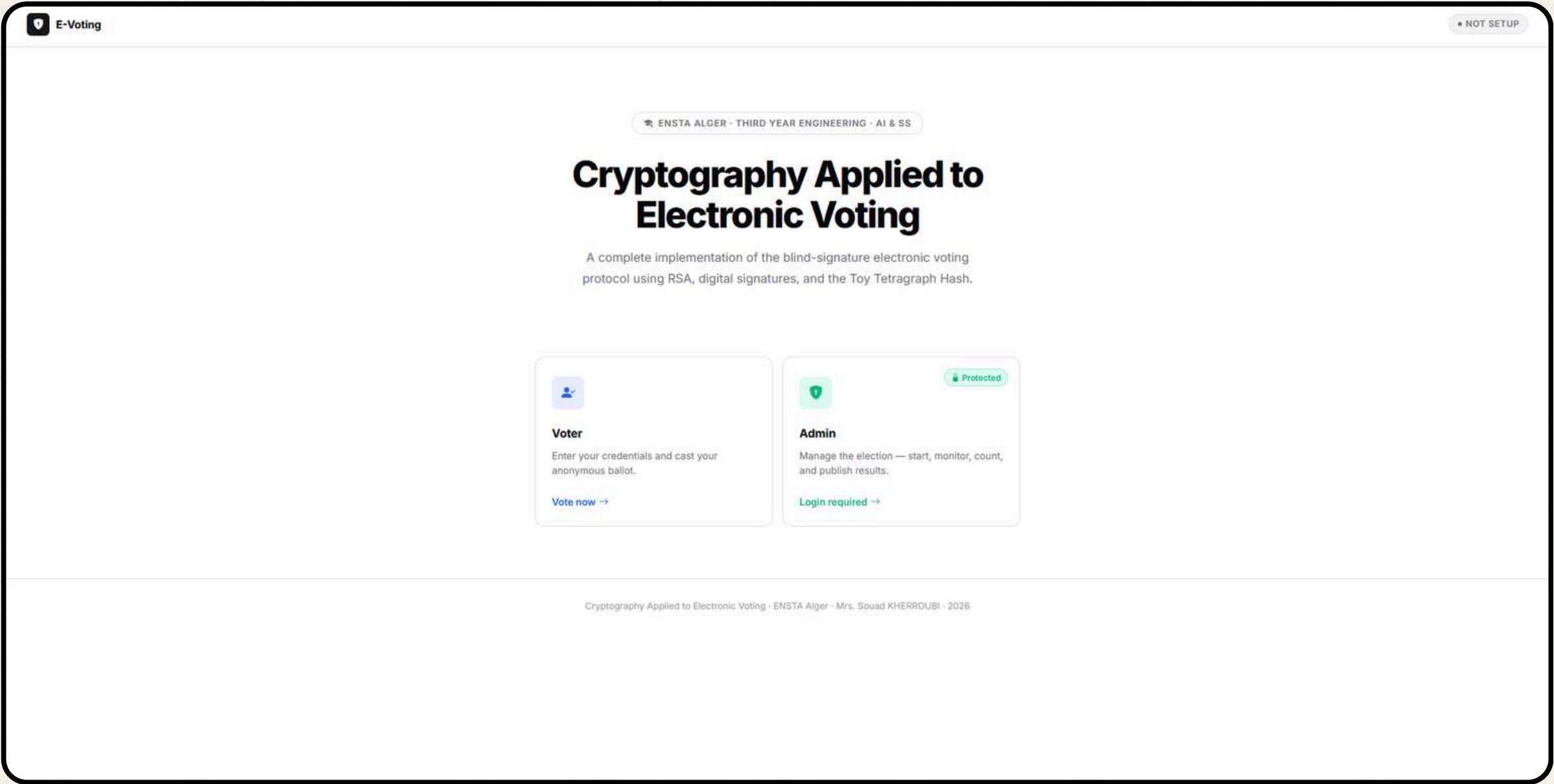
**Security Engineering:** The project provided essential hands-on experience in applying abstract mathematical concepts, such as prime numbers and modular arithmetic, to solve real-world security challenges.

**Defense in Depth:** We learned that security does not rely on a single algorithm but on a Distributed Trust Architecture that limits the power of every individual participant.



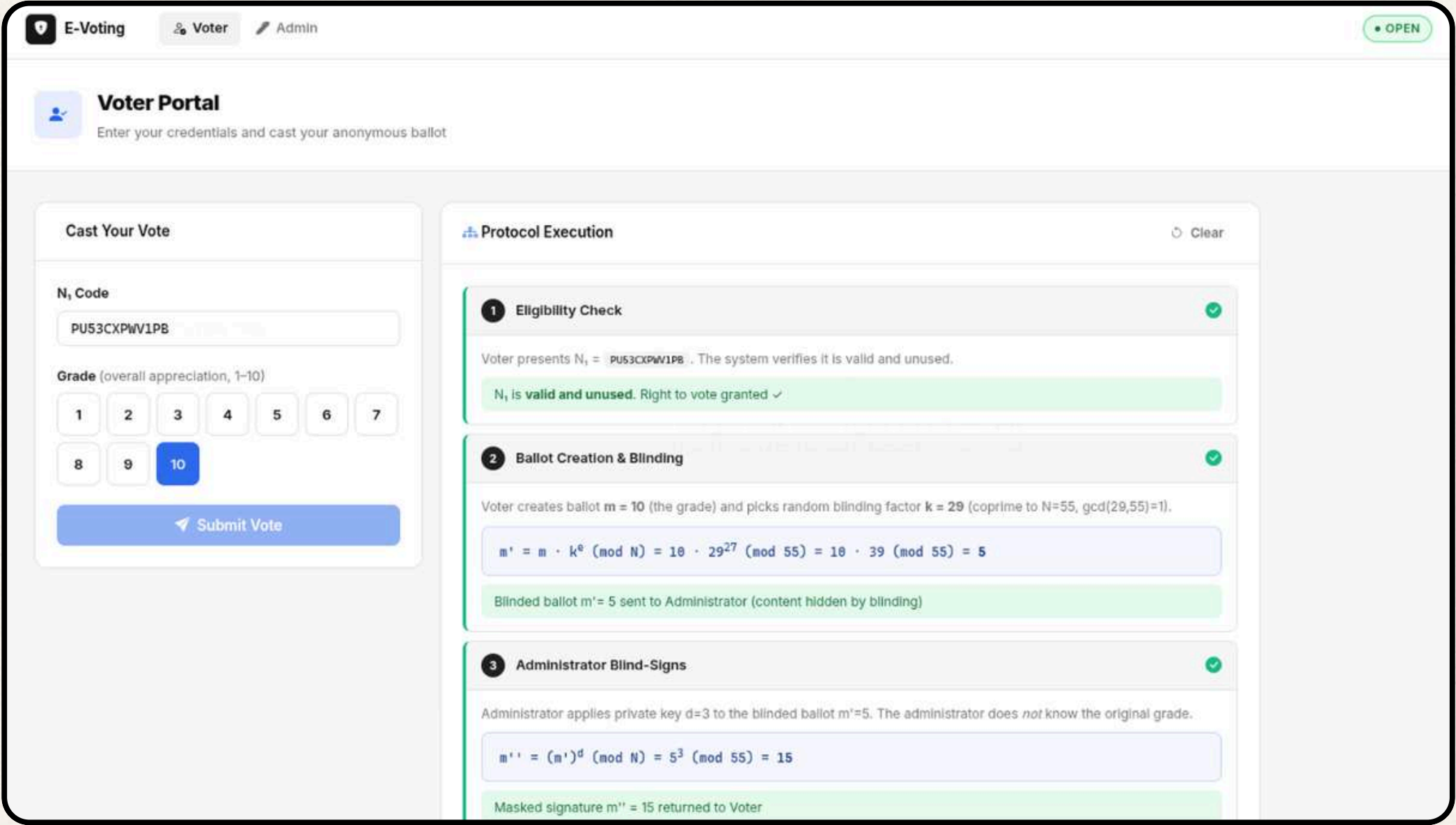


# Screenshots



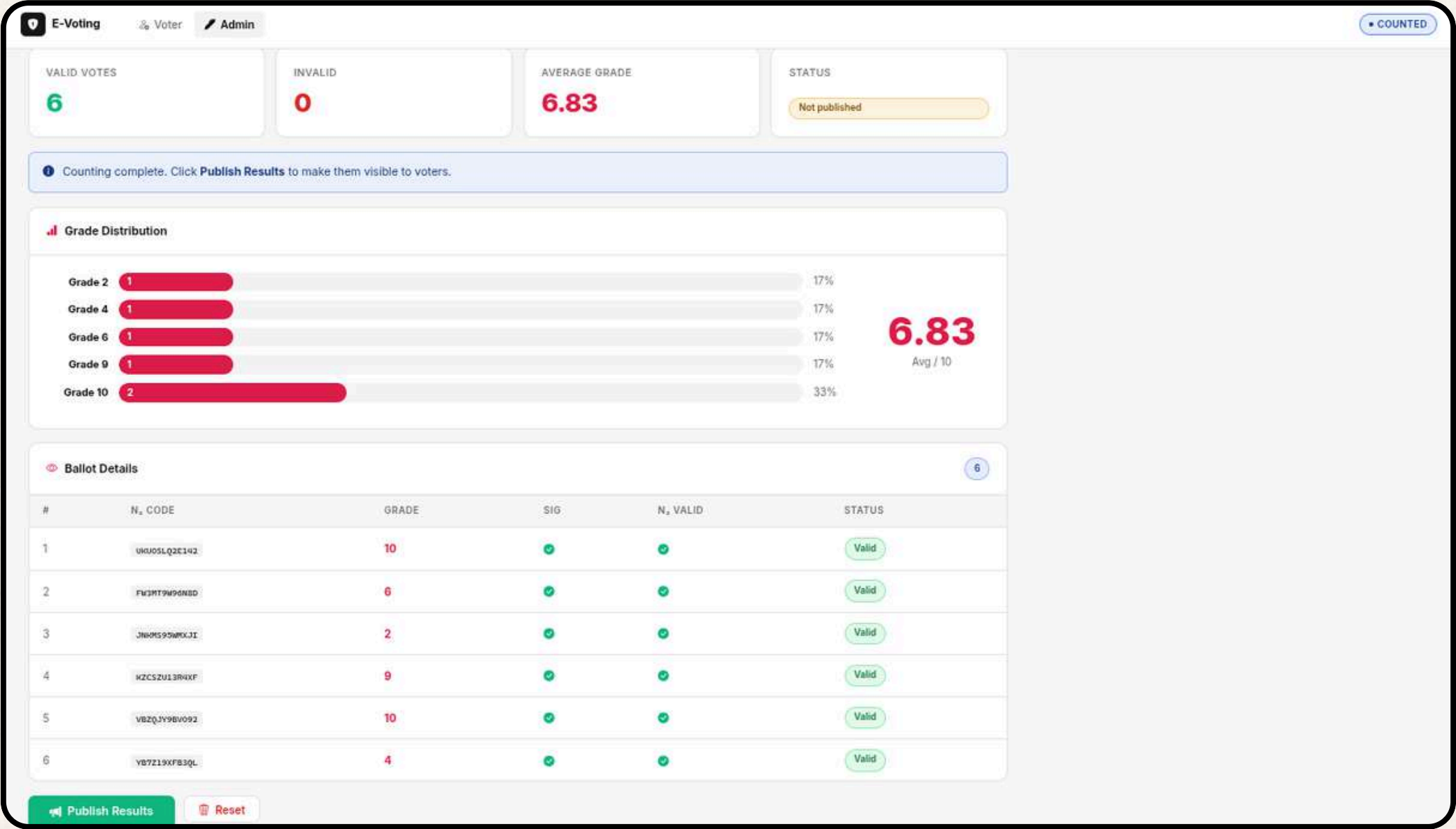


# Screenshots





# Screenshots





# Thank you

**PRESENTED BY:**

DAHMOUN Mouaine Aymen

BENCHENINA Shouhaib Meheni

BENZINA Mohamed

MEZAACHE Abdessamie

FANTAZI Sami Ilyes

**PRESENTED TO:**

Ms. KHERROUBI